# ASSIGNMENT 2 – CO2

# TASK 3: ENGINEER THE hashCode–equals CONTRACT

Problem Context: Event Attendance De-duplication

## Objective

Investigate how the contract between equals() and hashCode() affects duplicate detection in HashSet<Student>, design tests that reveal the behaviour, correct the implementation, and verify the result.

This report intentionally excludes student-identifying details.

# 1. Problem Understanding

A college fest scans student ID cards at multiple gates. A student may be scanned more than once. The system stores scans in HashSet<Student> so only unique students remain. The supplied class overrides equals() using regNo but does not override hashCode().

Contract: if two objects are equal according to equals(), they must return the same hashCode(). The reverse is not required: different objects may share a hash code.

# 2. Predictions Before Execution

**Test 1 – Same object reference inserted twice:** Insert the exact same object reference twice. Prediction: size 1, because HashSet recognizes the same object reference.

**Test 2 – Distinct duplicate objects in different buckets:** Create two separate objects with the same regNo but different identity-hash buckets. Prediction: size 2, because the broken class inherits identity-based hashCode() and HashSet searches only the bucket selected by that hash.

**Test 3 – Mutated key:** Insert an object whose hashCode() depends on regNo, then change regNo so its bucket changes. Prediction: contains() and remove() return false because the object remains stored in its original bucket.

# 3. Experimental Design and Results

The Java program constructs a distinct duplicate pair dynamically and selects two different bucket indexes in the initial 16-bucket table. It also performs the repeated-reference case and a mutable-key test. This avoids relying on a lucky collision of identity hash codes.

| Test | Design | Prediction | Observed |
|---|---|---|---|
| 1 | Same reference inserted twice | Size = 1 | Size = 1 |
| 2 | Equal regNo; distinct objects, different buckets | Size = 2 | Size = 2 |
| 3 | Change regNo after insertion | contains=false; remove=false | See console log |
| Corrected | hashCode() based on regNo | Unique students = 2 | Unique students = 2 |

Interpretation: Test 1 demonstrates that reinserting the same reference is naturally ignored; it does not prove that two distinct Student objects with the same regNo will be de-duplicated. Test 2 exposes the actual defect. The supplied incomplete three-insertion example can vary between executions because identity hash values and bucket placement are runtime-dependent.

# 4. How HashSet Uses hashCode() and equals()

HashSet obtains hashCode(), selects a bucket, and compares candidate entries in that bucket using equals(). If an equal candidate is found, the new object is not added. If equal objects produce different hash codes, they can be routed to different buckets and the equality check may not compare them.

## Why duplicates appear

The broken class inherits Object.hashCode(), which is based on object identity rather than regNo. Two separate Student objects with the same register number can therefore be directed to different buckets. Both may be stored.

## Mutation after insertion

If a field used in hashCode() changes after insertion, later contains() or remove() calls calculate a new bucket. The object may remain physically stored in its old bucket, making it difficult to find. Fields used for equality and hashing should remain immutable while stored in a hash-based collection.

## 5. Complete the Code – TODO 1

The supplied Student class defines equality by regNo. Complete TODO 1 with:

```
@Override
public int hashCode() {
    return regNo.hashCode();
}
```

The complete implementation is included in Attendance.java. The corrected class declares regNo final and uses the same field in equals() and hashCode().

### Expected output:

```
Unique students: 2
```

Run the supplied incomplete code before completing TODO 1 and record the output. Its simple three-insertion sample may display 2 or 3 depending on runtime identity hash values. The controlled experiments separately show the underlying reason.

## 6. Corrected Design and Verification

The corrected Student class treats regNo as the student's identity. equals() returns true for Student objects with the same regNo. hashCode() returns regNo.hashCode(). Equal objects therefore produce equal hash codes and are routed to the same bucket, where equals() can reject the duplicate. The corrected attendance example inserts Arun, Divya and Arun again and reports two unique students.

## 7. Alternative Concepts and Complexity

**TreeSet with Comparator on regNo:** A comparator that compares register numbers treats equal register numbers as the same set element when compare() returns zero. Add, lookup and removal are O(log n), with O(n) space.

**HashMap<String, Student> keyed by regNo:** Store the register number as the key and the Student as the value. Repeated keys update/replace the existing mapping. Put and get are O(1) average and O(n) worst case, with O(n) space.

| Structure | Insert | Lookup | Space |
|---|---|---|---|
| HashSet<Student> | O(1) average; O(n) worst | O(1) average; O(n) worst | O(n) |
| TreeSet<Student> | O(log n) | O(log n) | O(n) |
| HashMap<String, Student> | O(1) average; O(n) worst | O(1) average; O(n) worst | O(n) |

For n scans with u unique students, each structure stores O(u) entries. HashSet and HashMap provide average constant-time operations when hashes are well distributed. TreeSet provides sorted storage with logarithmic operations. HashMap keyed by regNo is a natural alternative when the register number is the key.

## 8. Conclusion

The tests show that equals() and hashCode() must be implemented as a consistent pair for hash-based collections. A class that overrides equals() but inherits identity-based hashCode() can appear to work in one case and fail for distinct duplicate objects. Using immutable regNo in both methods corrects the Student class and produces the expected two unique attendees.

## 9. Execution Procedure

Open a terminal in Task03. Compile with javac Attendance.java and execute with java Attendance. No keyboard input is required. Capture your own successful VS Code execution and save it in output_screenshots.

Evidence files: Attendance.java, task3_test_results.csv and run_output.txt. The PDF report intentionally excludes student-identifying details.